

MSC DISSERTATION DEFENSE

Improving Memory Safety in C Code with Nullability Analysis

Retrofitting Rust-inspired, compile-time null-pointer safety onto standard C.

Francisco Miguel Correia Mariano Pinheiro Cardoso

Supervisors: Prof. João Bispo · Prof. Tiago Carvalho

July 2026

OUTLINE

01 Introduction

The problem, and why it matters

03 Null Safety Model

The theory: dataflow and contracts

05 Evaluation

Two real codebases

02 Related Work

What exists

04 Implementation

The nulck module inside CORAL

06 Conclusions

Answers, limitations, future work

C still runs our most critical systems

- Operating systems, drivers, embedded — still C
- Precise hardware control, no runtime overhead
- The cost: memory is managed entirely by hand

WHY IT MATTERS

Memory-safety violations
account for the vast majority of
exploited security flaws.

From simple crashes to catastrophic breaches in life-critical systems.

Three ways manual memory goes wrong

FOCUS OF THIS WORK

Null pointer dereference

Using a pointer that points to nothing — an immediate crash.

TEMPORAL

Use-after-free

Accessing memory after it was freed — corruption and exploits.

RESOURCE

Memory leak

Allocated memory never freed — the system slowly starves.

Rust is safe by construction — but rewriting is not an option

RUST'S GUARANTEE

- ✓ Ownership and borrowing rules
- ✓ Enforced by the compile-time borrow checker
- ✓ No garbage collector, no runtime cost

THE CATCH

- ✗ Legacy C is millions of lines of code
- ✗ Rewrites are costly and risk regressions
- ✗ Established build and test infrastructure

The goal: retrofit safety into C — incrementally, one component at a time.

CORAL — Rust-inspired safety, retrofitted into C

- Source-to-source, built on the Clava compiler
- Analyses a CFG from the high-level AST — no semantic gap
- Standard C stays standard C
- Prior work: non-lexical lifetimes, move analysis

THE MISSING PIECE

Earlier CORAL had no null-pointer semantics, no multi-level pointers, and no rigorous interprocedural boundaries.

Research questions

RQ1 Nullability modeling

Model null-pointer state without breaking C idioms.

RQ2 Interprocedural boundaries

Enforce safety across calls without full source visibility.

RQ3 Real-world applicability

How much more real C code can CORAL now verify?

RQ4 Workflow integration

Stay compatible with C standards; adopt incrementally.

PART 03

Null Safety Model for C

The theoretical blueprint: a flow-sensitive dataflow analysis, and a modular interprocedural contract system.

Two pillars of the model

01

Flow-sensitive dataflow

Traverse a control-flow graph and simulate the real execution paths — branches, loops, early exits.

→ local precision

02

Interprocedural contracts

Analyse each function in strict isolation, relying only on contracts attached to its signature.

→ scalability

A pointer has three possible states

NOT_NULL

Guaranteed valid — safe to dereference.

MAYBE_NULL

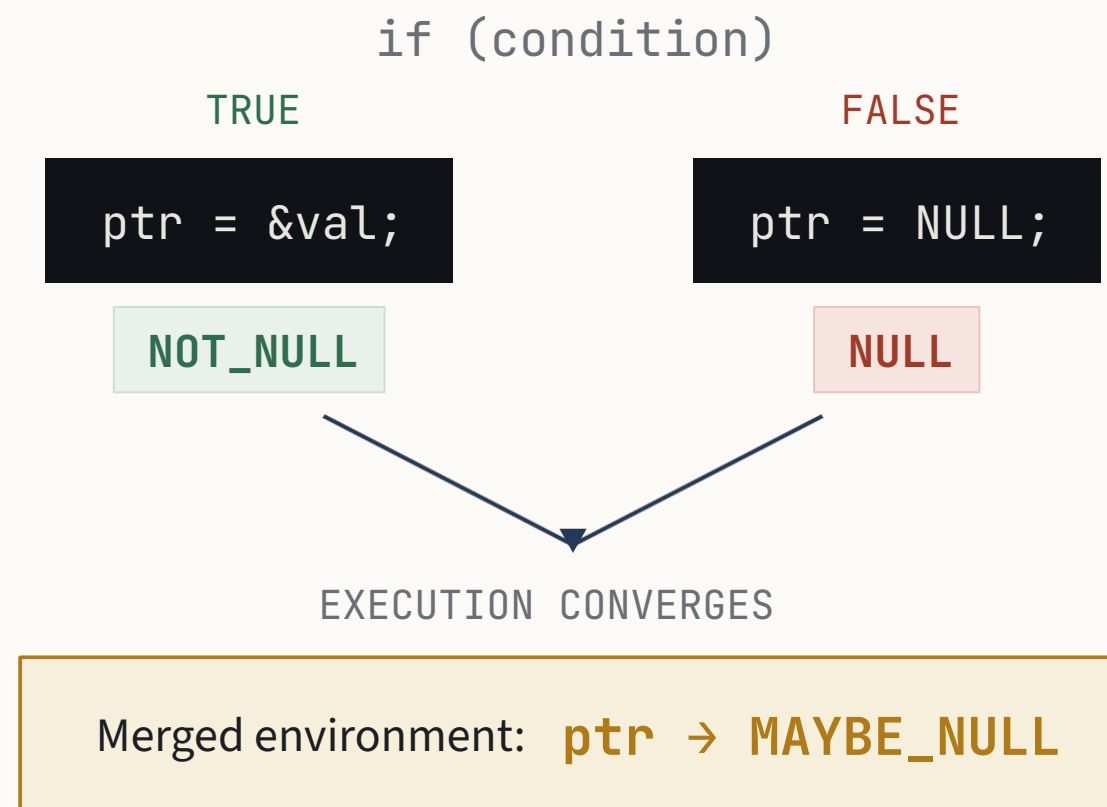
Unknown — must be checked before use.

NULL

Definitely points to nothing.

rule When paths disagree, the state conservatively downgrades to **MAYBE_NULL**.

States merge where paths converge



Guards refine state along each branch

```
void process_data(int *ptr) {  
    // ptr is MAYBE_NULL  
    if (ptr ≠ NULL) {  
        // refined → NOT_NULL  
        int v = *ptr;  
    } else {  
        return; // early exit  
    }  
    *ptr = 10; // safe: still NOT_NULL  
}
```

- ✓ True branch → **NOT_NULL**
- ✓ False branch → **NULL**
- An early return prunes the unsafe path — the dereference below is proven safe.

Loops resolve by fixed-point iteration

- Loop back-edges create cycles in the CFG
- Re-analyse the body, merging state on each pass
- Stop when the environment stops changing — the **fixed point**
- `break` and `continue` fall out naturally from the graph

Guarantees the analysis always **terminates** — even for deeply nested loops.

Normalization tames C's many syntaxes

Switch, ternary, && and || are all rewritten to one structure: nested `if-else`.

SHORT-CIRCUIT IDIOM

```
if (ptr ≠ NULL && *ptr > 0) {  
    process_data(ptr);  
}
```



NORMALIZED – SAFE BY CONSTRUCTION

```
if (ptr ≠ NULL) {  
    //NOT_NULL  
    if (*ptr > 0) {  
        process_data(ptr);  
    }  
}
```

Crossing function boundaries with contracts

Analyse every function in isolation — describe its boundary with four primitives.

01 • ENTRY

Pre-condition

State a parameter must hold before the call.

02 • EXIT

Post-condition

Guaranteed state after the function returns.

03 • UNCHANGED

Invariant

The function will not mutate the state.

04 • RETURN

Return state

State of the address handed back — e.g. malloc.

Result: complexity scales linearly with functions, not exponentially with call depth.

Modeling deep and structured memory

MULTI-LEVEL POINTER – `int **ptr`

`ptr` NOT_NULL

↓ dereference

`*ptr` MAYBE_NULL

↓ dereference

`**ptr` (integer value)

FIELD-SENSITIVE STRUCT

`obj` NOT_NULL

`obj→field1` MAYBE_NULL

`obj→field2` NOT_NULL

Each topological level is tracked **independently** — states are never conflated.

Honest limits: globals, liveness & moves

GLOBALS

Invalidate on external calls

Any call may mutate a global — downgrade to **MAYBE_NULL**. Assumes single-threaded execution.

NULLABILITY ≠ LIVENESS

Dangling but not null

A pointer to freed / out-of-scope memory is still **NOT_NULL** here — liveness is a separate analysis.

NULLABILITY ≠ OWNERSHIP

Moved but not null

A moved pointer still physically holds its address — use-after-move belongs to move analysis.

A deliberate separation of concerns — each analysis stays orthogonal and sound.

PART 04

Implementation: the nulck module

From theoretical model to a working analyzer, integrated into CORAL.

nulck runs as an isolated module

- Earlier CORAL ran all checkers together
- nulck is fully decoupled — run it on its own
- Consumes normalized output; keeps its own environment
- Users toggle exactly the checks they care about

FAIL-EARLY INTERFERENCE

When checkers run together, a borrow error aborts compilation
— and the null bug on the next line is never even reached.

Contracts are written as `#pragma coral`

```
#pragma coral ptr : maybe-null → not-null
#pragma coral ensures return not-null
#pragma coral read_only_ptr : unchanged

// nested { } – structs & double pointers
#pragma coral obj {field1: maybe-null → not-null}

// %(regex) – variadic / bulk rules
#pragma coral %(.ptr.*) : not-null
```

→

left = entry, right = exit

{ }

target fields & indirection levels

%()

bulk rules for variadic calls

Standard compilers **ignore** unknown pragmas — code still builds.

The engine

HOW IT WORKS

- per-node in / out environments
- Fixed-point check halts cyclic traversal
- Condition nodes clone the environment per branch

THREE VIOLATION CLASSES

NullDereferenceError

Dereferencing a null / maybe-null pointer.

PreconditionViolationError

Argument fails the callee's entry state.

ContractViolationError

A function breaks its own exit promise.

PART 05

Evaluation & Results

Unit tests, an amateur C program, and a high-performance benchmark.

Evaluated in three progressive stages

01

Unit tests

Dozens of edge cases across four behavioural categories.

02

Amateur C program

Unstructured, real-world legacy code.

03

PolyBench/C

Optimised high-performance computing code.

All source publicly available for reproducibility.

Case study: uncovering real bugs

A contractual **ripple effect** pushes null-checking up the call stack — until the root bug appears.

PHASE 1

Annotate libraries

Contracts for strtol, read.

PHASE 2

Propagate inward

Helpers must declare their preconditions.

PHASE 3

False positives

argv needs a manual check — no OS knowledge.

PHASE 4

True bugs

Unchecked malloc → averted segfault.

A rigid safety net for novices — it forces defensive programming habits.

Benchmark: auditing optimised HPC code

- `atax.c` — matrix / vector kernel, heavy pointer arithmetic
- Default: pessimistic — internal dereferences flagged
- Add `NOT_NULL` contracts → burden shifts to `main`
- HPC anti-pattern exposed: `malloc` never checked

THE VALUE

A strict auditor, even for expert code.

Verifies memory safety and enforces defensive-programming discipline where checks are usually stripped for speed.

PART 06

Conclusions

Answers to the research questions, current limitations, and future work.

Limitations — where nulck stops

TEMPORAL SAFETY

The free() conundrum

free leaves a dangling pointer that still reads as NOT_NULL — use-after-free is missed.

SPATIAL SAFETY

No bounds awareness

A non-null pointer says nothing about size — buffer overflows are missed.

USABILITY

Annotation burden

Every standard-library function needs a contract today.

Contributions

- 01 **nulck** — a static nullability module for CORAL
 - 02 A path-sensitive dataflow engine
 - 03 An interprocedural contract system via `#pragma` annotations
 - 04 Evaluation on amateur code and benchmarks
-

Answering the Research questions

RQ1 Nullability modeling

Model null-pointer state time without breaking C idioms.

RES : Lattice + path-sensitive over the CFG — idioms preserved.

RQ2 Interprocedural boundaries

Enforce safety across calls without full source visibility.

RES: Contracts + conservative invalidation — no whole-program cost.

RQ3 Real-world applicability

How much more real C code can CORAL now verify?

RES: Found genuine bugs in both amateur and HPC code.

RQ4 Workflow integration

Stay compatible with C standards; adopt incrementally.

RES: #pragma is ignored by compilers — incremental adoption.

Thank you.

Questions are very welcome.

Francisco Cardoso

Supervisors: Prof. João Bispo · Prof. Tiago Carvalho

Improving Memory Safety in C with Nullability Analysis

Future work — toward full safety

TEMPORAL

Ownership states

free consumes the pointer → INVALID. Tags
@consumes / @borrows ; automatic scope
drops.

SPATIAL

Bounds tracking

Track a size / bound per pointer;
@expects_size(N) ; verify arithmetic and
indexing.

USABILITY

Built-in registry

Ship standard-library contracts by default — no
header pollution.